Indian Institute of Technology Madras

DA5402 – MLOps

Project Report

MLOps Review Pulse

Submitted By

Apurv Ravindra Kshirsagar

CE22B042

April 28, 2026

Contents

1 Abstract **3**

2 Introduction **3**

3 Problem Statement **3**

4 Objectives **4**

5 System Overview **4**

6 Technology Stack **5**

7 Dataset Description **5**

8 Data Preprocessing **6**

9 Model Development **6**

9.1 Baseline Model: TF-IDF + Logistic Regression 6

9.2 Advanced Experiment: DistilBERT 6

9.3 Evaluation Metrics 7

10 Frontend Application **7**

10.1 Single Review Prediction 7

10.2 Bulk CSV Upload 7

10.3 Analytics Dashboard 8

10.4 Pipeline Overview Page 8

11 Backend API Design **9**

12 Workflow Automation with Airflow **10**

13 Experiment Tracking with MLflow **10**

14 Monitoring and Observability **12**

14.1 Prometheus 12
14.2 Grafana 12

15 Automated Alerting **12**

16 Testing and Validation **13**

17 Challenges Faced **14**

18 Conclusion **14**

19 References **15**

1. Abstract

MLOps Review Pulse is an end-to-end machine learning operations platform developed for customer review sentiment analysis. The system accepts user reviews through an interactive web interface or CSV upload, predicts sentiment classes, tracks model experiments, automates data workflows, monitors APIs in real time, and generates alerts for operational issues such as data drift or missing fresh data.

The objective of this project is not limited to building a machine learning model. Instead, it demonstrates the complete production lifecycle of an ML system using modern tools such as FastAPI, Docker Compose, Apache Airflow, MLflow, Prometheus, Grafana, DVC, and automated testing frameworks. The final solution emphasizes reproducibility, modularity, observability, and maintainability.

2. Introduction

Organizations receive large volumes of textual feedback through e-commerce platforms, service portals, application stores, and social media. These reviews contain valuable information regarding customer satisfaction, product defects, usability, and service quality. Extracting insights manually from thousands of reviews is expensive and inefficient.

Sentiment analysis is a Natural Language Processing (NLP) task that classifies text into sentiment categories such as positive or negative. However, a model alone is not sufficient in real-world scenarios. Production systems require data pipelines, retraining workflows, deployment strategies, monitoring dashboards, alerting systems, testing, and version control.

This project was developed to address that broader requirement. MLOps Review Pulse converts a traditional sentiment classifier into a complete production-grade machine learning platform.

3. Problem Statement

Most academic machine learning projects focus only on training and accuracy metrics. In real deployments, additional challenges arise:

- How is new data ingested automatically?
- How are experiments tracked and compared?

- How are trained models versioned and deployed?
- How is system health monitored continuously?
- How are failures detected and notified?
- How can the same system run consistently across machines?
- How is software quality ensured through testing?

The problem addressed in this project is the design and implementation of a complete MLOps pipeline for review sentiment analysis.

4. Objectives

1. Build an accurate sentiment classification system for review text.
2. Develop REST APIs for single, batch, and CSV predictions.
3. Create a user-friendly dashboard for interacting with the model.
4. Automate ingestion and preprocessing pipelines using Airflow.
5. Track experiments and register models using MLflow.
6. Containerize the entire platform using Docker Compose.
7. Monitor performance using Prometheus and Grafana.
8. Configure automated alerting for anomalies.
9. Validate system behavior using automated tests.

5. System Overview

The platform is composed of multiple services working together:

- **Frontend:** User interface for predictions and analytics.
- **Backend API:** FastAPI service for inference and metrics.
- **Model Layer:** Trained sentiment classifier.
- **Airflow:** Workflow orchestration.

- **MLflow:** Experiment tracking and model registry.
- **Prometheus:** Metrics collection.
- **Grafana:** Dashboard visualization.
- **Alertmanager:** Email alerts.
- **Docker Compose:** Multi-service deployment.

6. Technology Stack

Technology	Purpose
Python	Core programming language
FastAPI	REST API development
React / Frontend UI	User dashboard and interactions
Scikit-learn	TF-IDF + Logistic Regression model
Transformers	DistilBERT experiments
Apache Airflow	Data pipeline orchestration
MLflow	Experiment tracking and registry
Docker Compose	Containerized deployment
Prometheus	Metrics scraping
Grafana	Visualization dashboards
Alertmanager	Notifications
Pytest	Automated testing
DVC	Data versioning

7. Dataset Description

The project uses sentiment-labeled review data collected from common domains such as product reviews, restaurant reviews, and movie reviews. Each record contains review text and a binary label representing sentiment.

Typical Fields:

- Review text
- Sentiment label (Positive / Negative)

The dataset was cleaned, standardized, and used for model training and evaluation.

8. Data Preprocessing

Raw text cannot be directly used by machine learning algorithms. The following preprocessing steps were applied:

- Lowercasing text
- Removing unwanted symbols
- Whitespace normalization
- Handling missing values
- Feature extraction using TF-IDF

These steps improve consistency and help the model learn meaningful textual patterns.

9. Model Development

9.1 Baseline Model: TF-IDF + Logistic Regression

The production baseline model uses TF-IDF vectorization followed by Logistic Regression.

Reasons for Selection:

- Fast training time
- Strong performance on short text
- Lightweight deployment
- Easy interpretability

9.2 Advanced Experiment: DistilBERT

Transformer-based models were also explored for improved semantic understanding. DistilBERT provided stronger contextual modeling but required more compute resources.

9.3 Evaluation Metrics

The following metrics were used:

- Accuracy
- Precision
- Recall
- F1-score

10. Frontend Application

The web interface allows users to interact with the system visually.

10.1 Single Review Prediction

Users can enter a single review and obtain sentiment with confidence.

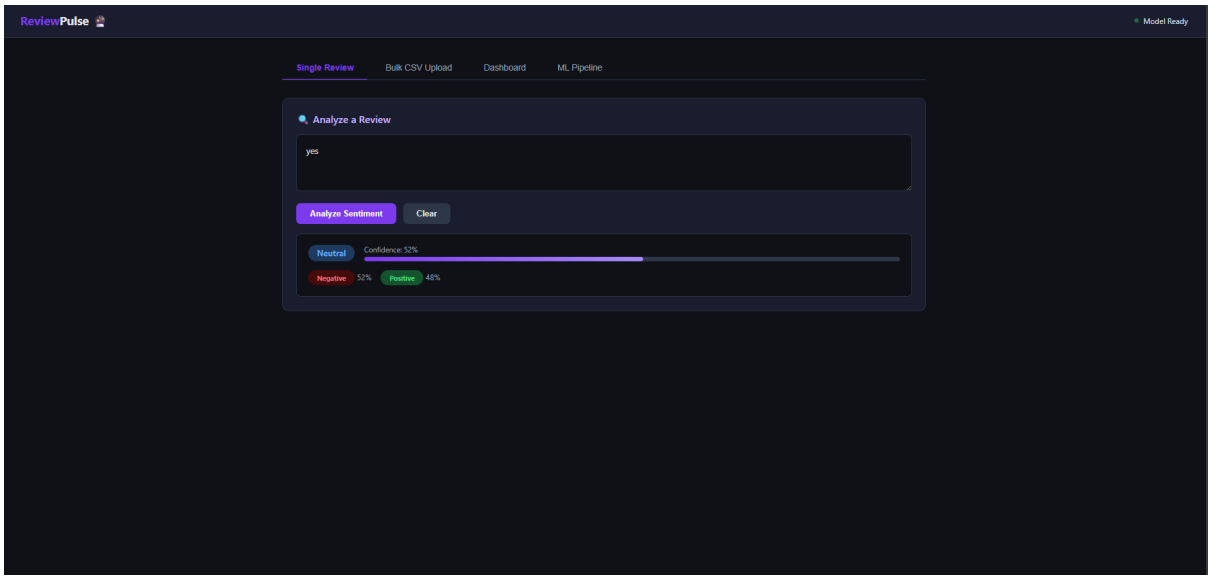

Figure 1: Single Review Prediction Interface

10.2 Bulk CSV Upload

Users can upload a CSV file containing multiple reviews and receive batch predictions with downloadable results.

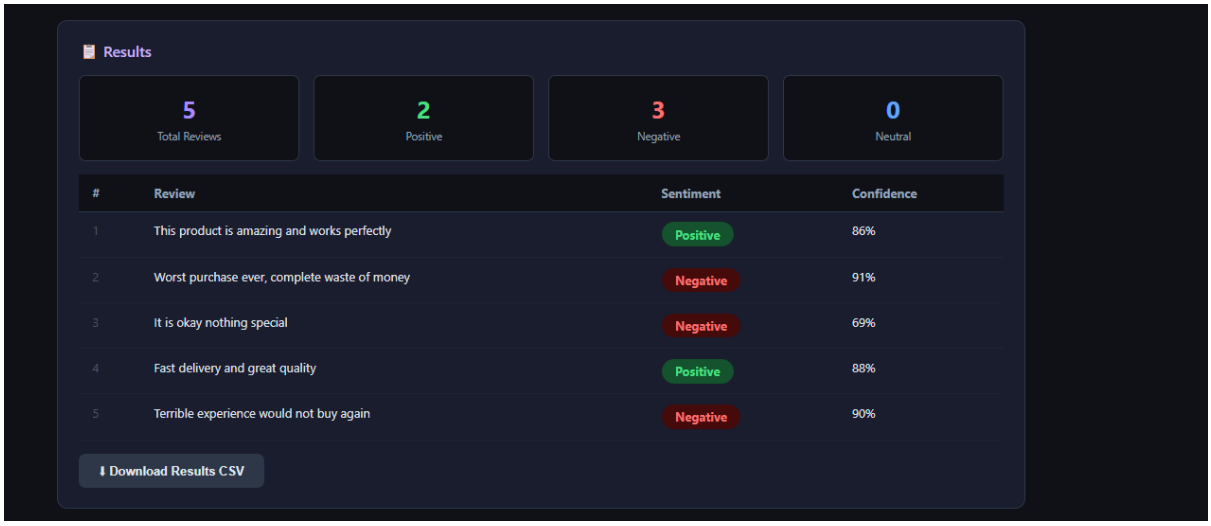

Figure 2: Bulk CSV Prediction Results

10.3 Analytics Dashboard

The dashboard summarizes positive rate, negative rate, average confidence, and keyword insights.

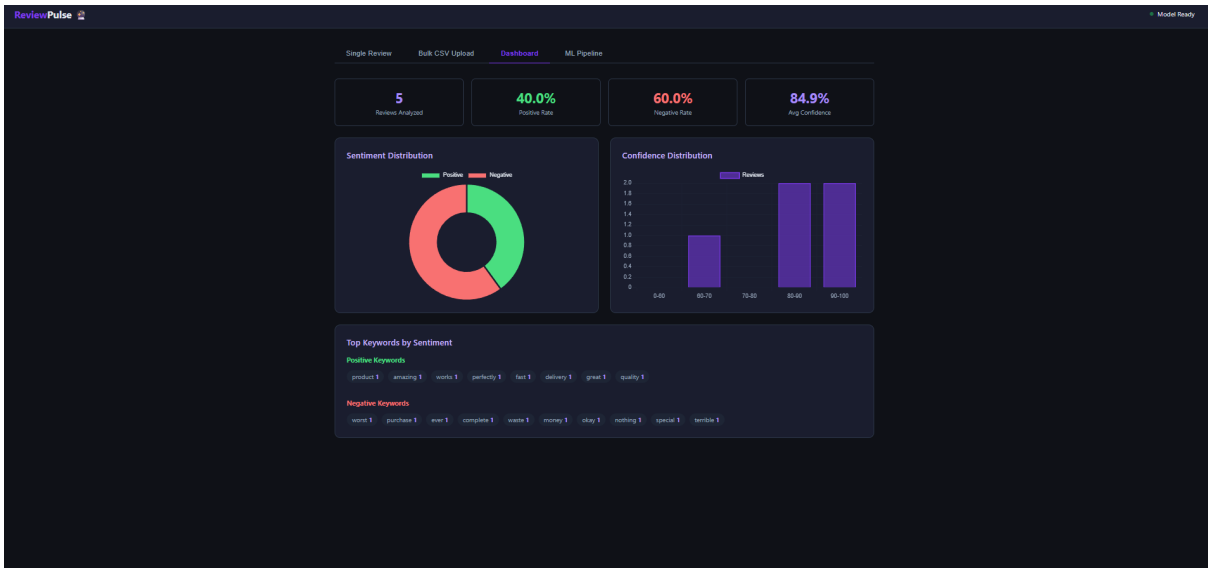

Figure 3: Frontend Analytics Dashboard

10.4 Pipeline Overview Page

The application also presents pipeline stages, API endpoints, and technology stack.

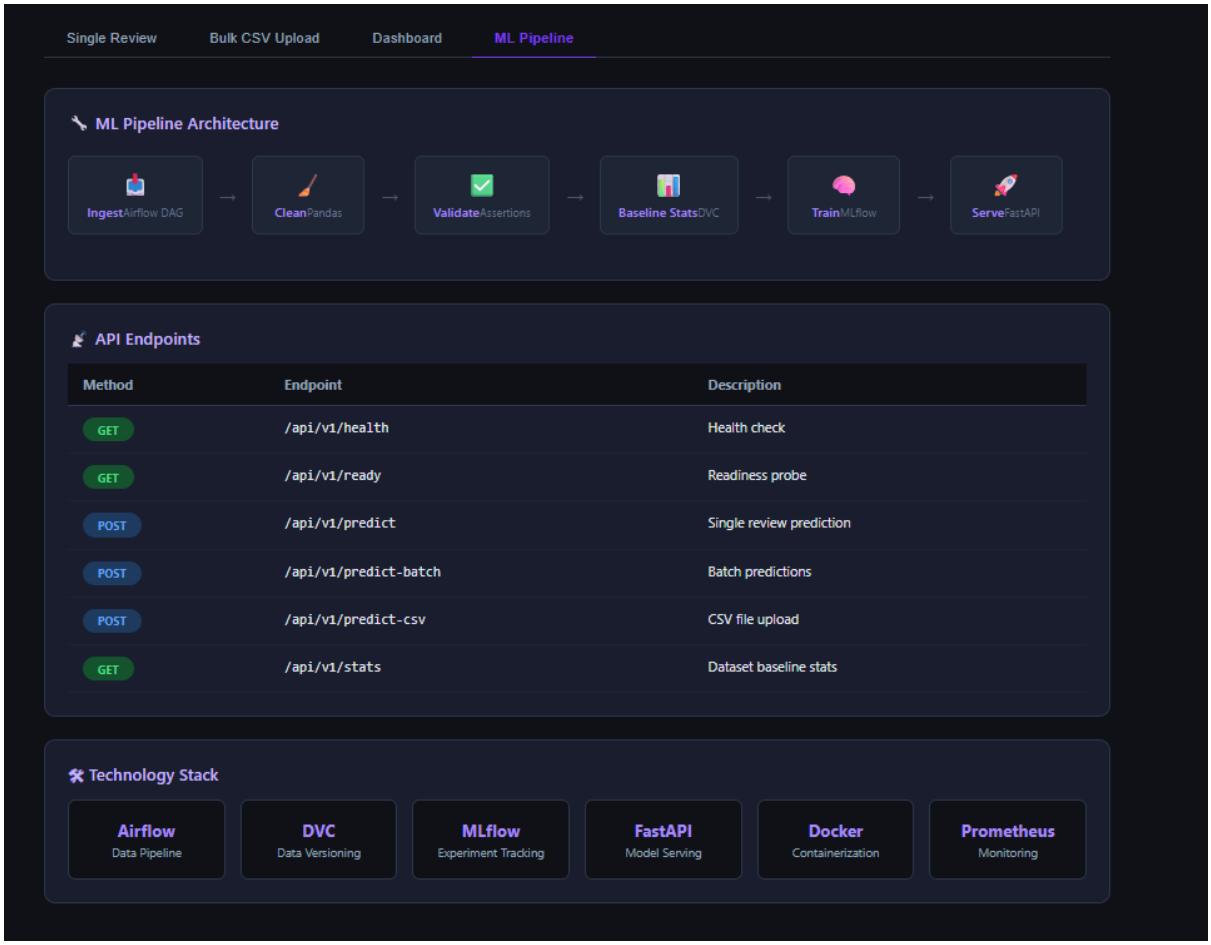

Figure 4: ML Pipeline Overview in Frontend

11. Backend API Design

The FastAPI backend exposes production-ready endpoints.

Method	Endpoint	Purpose
GET	/api/v1/health	Health check
GET	/api/v1/ready	Readiness probe
POST	/api/v1/predict	Single prediction
POST	/api/v1/predict-batch	Batch prediction
POST	/api/v1/predict-csv	CSV upload predictions
GET	/metrics	Prometheus metrics

This separation of endpoints enables easy integration with frontend systems, automation scripts, and monitoring tools.

12. Workflow Automation with Airflow

Apache Airflow orchestrates the machine learning data pipeline. The DAG automates sequential tasks such as:

1. Ingest raw data
2. Clean dataset
3. Validate schema and quality
4. Compute baseline statistics
5. Detect drift
6. Trigger training

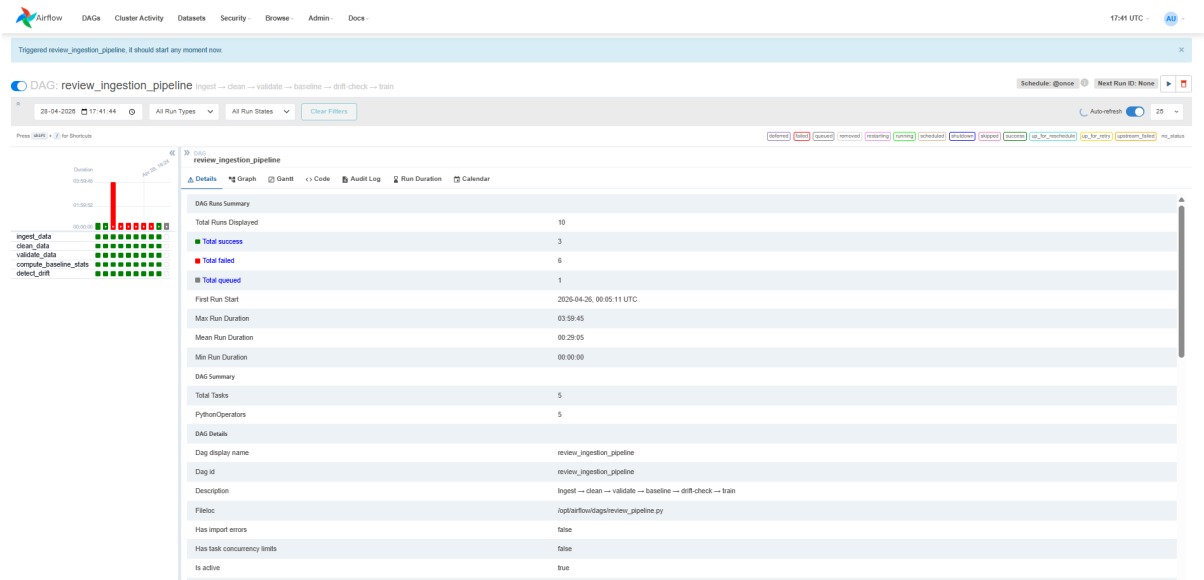

Figure 5: Airflow DAG Execution Interface

Airflow removes manual repetition and ensures scheduled reproducible execution.

13. Experiment Tracking with MLflow

MLflow is used to log experiments, metrics, parameters, and model artifacts.

Tracked Information:

- Accuracy

- F1-score
- Train/Test size
- Hyperparameters
- Model versions

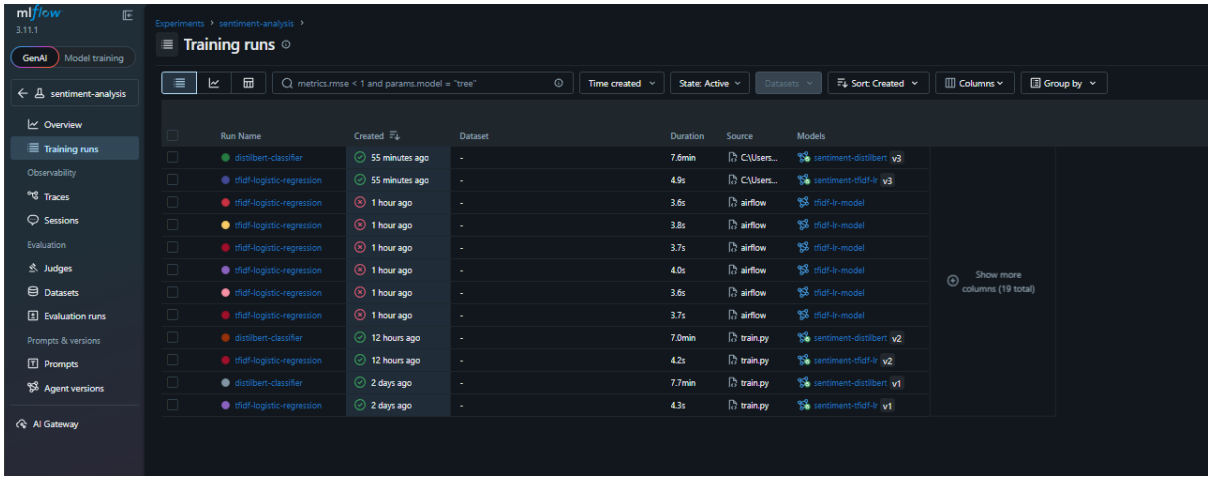

Figure 6: MLflow Training Runs

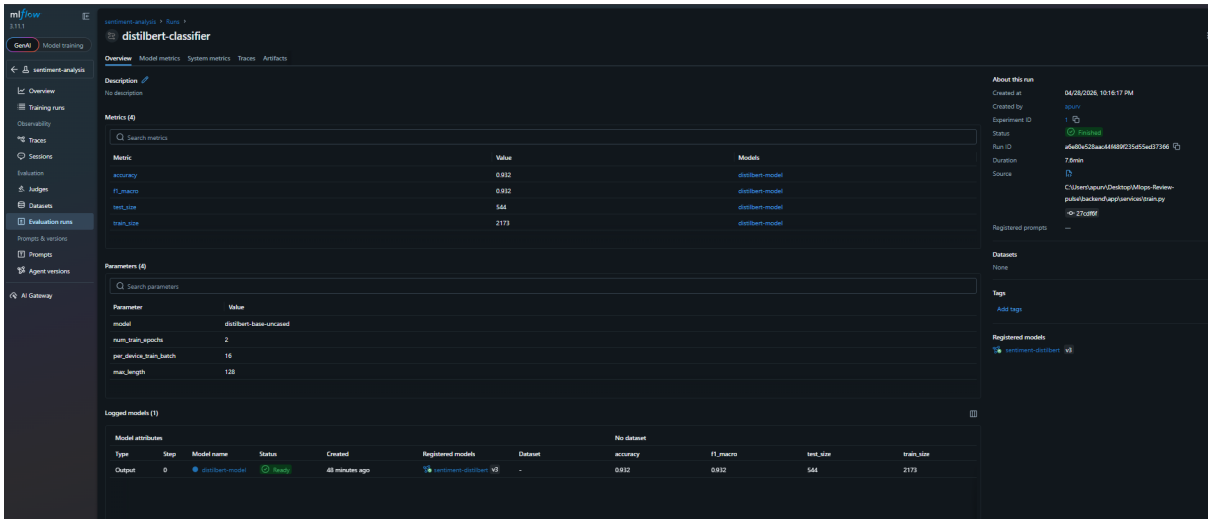

Figure 7: Detailed MLflow Run Metrics

This enables systematic comparison between models and reproducible experimentation.

14. Monitoring and Observability

14.1 Prometheus

Prometheus collects backend metrics such as request counts, latency, and error rate.

14.2 Grafana

Grafana visualizes metrics in dashboards for operational visibility.

Figure 8: Grafana Monitoring Dashboard

Real-time observability is essential for production systems.

15. Automated Alerting

Operational issues should trigger immediate notifications. Alertmanager was configured to send email alerts for:

- Data drift detected
- Dry pipeline (no fresh data)
- Service failures

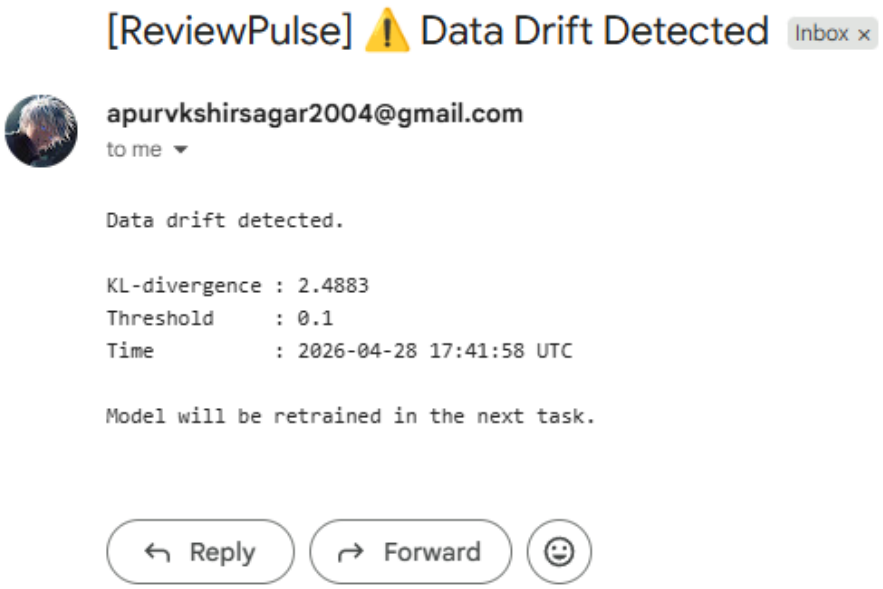

Figure 9: Email Alert for Data Drift

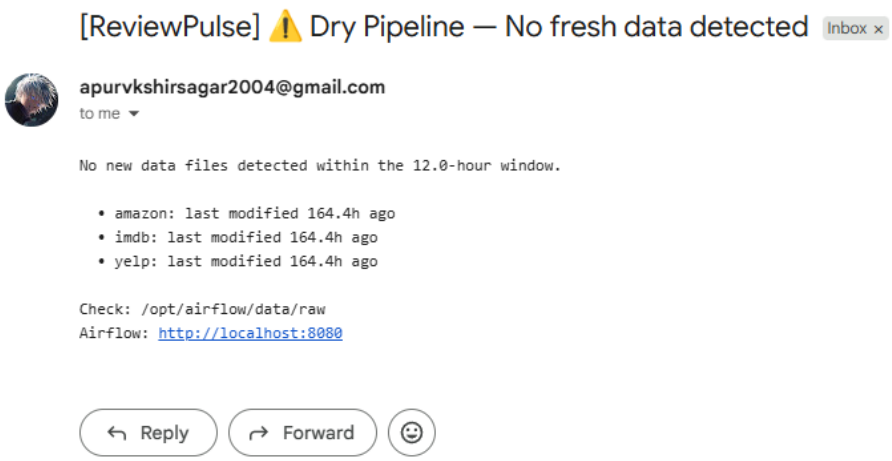

Figure 10: Email Alert for Dry Pipeline

This reduces reaction time and improves operational reliability.

16. Testing and Validation

Automated tests were developed using Pytest to validate the application.

Covered Areas:

- API health checks

- Single predictions
- Batch predictions
- CSV uploads
- Metrics endpoints
- Alerting utilities
- Configuration checks

Testing ensures code quality and helps prevent regressions during development.

17. Challenges Faced

Several real engineering challenges were encountered:

- Docker networking and service communication
- MLflow host header security configuration
- Cross-platform Windows/Linux path handling
- Container startup delays due to dependencies
- Test environment mismatch between local and Docker runs
- Monitoring configuration syntax issues

Resolving these issues improved the robustness of the final system.

18. Conclusion

MLOps Review Pulse demonstrates how a machine learning solution can be transformed into a production-ready platform. The project integrates model development, API serving, workflow orchestration, experiment tracking, monitoring, alerting, testing, and containerized deployment into one unified system.

The key learning from this work is that successful machine learning systems depend not only on model accuracy, but also on reliability, reproducibility, maintainability, and observability. This project serves as a practical implementation of modern MLOps principles.

19. References

1. FastAPI Documentation: https://fastapi.tiangolo.com
2. Apache Airflow Documentation: https://airflow.apache.org
3. MLflow Documentation: https://mlflow.org
4. Docker Documentation: https://docs.docker.com
5. Prometheus Documentation: https://prometheus.io
6. Grafana Documentation: https://grafana.com
7. Scikit-learn Documentation: https://scikit-learn.org